

**POSTS AND TELECOMMUNICATIONS
INSTITUTE OF TECHNOLOGY
FACULTY OF INFORMATION TECHNOLOGY 1**



ACADEMIC DESIGN REPORT

**FROM ANALYSIS
TO DESIGN**

MVC and DAO Class Design for an E-Commerce System

Student:	Vu Dinh Hieu
Student ID:	B23DCCE036
Design artifact:	eComDesign.vpp
Course topic:	Information Systems Analysis and Design

September 2026

Abstract

This report documents the transition from an analysis-oriented e-commerce class model to a layered object-oriented design. The resulting Visual Paradigm artifact organizes the system into a presentation interface, controllers, and domain model. Data access is expressed through DAO interfaces and repository implementations, allowing controllers to depend on contracts rather than persistence details. The design addresses four principal functional areas: customer management, shopping carts, item catalog browsing, and order processing. The final class diagram is included as the central design artifact and is interpreted through responsibilities, dependencies, and representative interaction paths.

The report also distinguishes structural verification from executable validation. The project was opened and exported with Visual Paradigm; the final diagram contains 35 class shapes, no overlapping class rectangles within the same package, and no broken diagram connector references. These checks establish that the edited artifact is structurally coherent and readable. They do not constitute a running Spring application or a proof of all business rules.

Keywords: e-commerce; UML class diagram; MVC; Data Access Object; layered architecture; analysis-to-design transformation.

Contents

Abstract	i
1 Introduction	1
1.1 Problem context	1
1.2 Objectives and scope	1
2 Design Foundations	2
2.1 From analysis classes to design classes	2
2.2 MVC and DAO	2
3 E-Commerce Class Design	4
3.1 Overview of the final diagram	4
3.2 Domain organization	6
3.3 Representative interactions	6
4 Evaluation and Limitations	7
4.1 Structural verification	7
4.2 Interpretive limits and further work	7
5 Conclusion	8

Chapter 1

Introduction

1.1 Problem context

An analysis model identifies domain concepts and their relationships. A design model must additionally assign software responsibilities, make dependencies explicit, and define boundaries between interaction, business state, and persistence. In the supplied course handout, the e-commerce system is used to demonstrate this progression: methods are added to classes, selected associations are replaced by use dependencies, DAO interfaces are introduced, and the classes are organized into an MVC-oriented architecture [1].

The design presented here is a class-level view of that progression. It is intended to communicate how the e-commerce application could be structured, rather than to prescribe a database schema or claim that a deployed application already exists. The diagram is maintained in the editable Visual Paradigm project `eComDesign.vpp`; Figure 3.1 reproduces its final exported view.

1.2 Objectives and scope

The objectives are to separate presentation from domain data, place request coordination in controllers, provide explicit persistence contracts for the main aggregates, and retain traceability to the original customer, cart, order, and item concepts. The scope of this report is the UML design artifact and its architectural interpretation. Operational concerns such as authentication, payment-provider integration, transactional behavior, and deployment are identified as further design work rather than silently assumed to be complete.

Chapter 2

Design Foundations

2.1 From analysis classes to design classes

In analysis, a class can be understood primarily as a business concept. In design, its operations and dependencies must express a realizable responsibility. For example, a `Cart` stores cart state and exposes operations related to adding or removing items and calculating a total. A controller coordinates the request that invokes those operations, while a DAO retrieves or saves the corresponding data. This allocation prevents a single class from carrying user-interface, business, and persistence responsibilities simultaneously.

The handout distinguishes methods intrinsic to an object from methods that use another class [1]. UML use dependencies in the design therefore indicate that one element requires another element's public contract. They should not be interpreted as database foreign keys or as proof of a complete runtime call sequence. Associations and inheritance remain appropriate where the domain relationship itself is the information being represented.

2.2 MVC and DAO

The Model–View–Controller pattern divides interaction into complementary concerns. A view presents information and accepts user input; a controller receives or coordinates a user action; the model represents domain state and rules. The diagram's `interface` (view), `controller`, and `model` packages make these responsibilities visible. In an implementation, a controller may call a view method to present a result, while user events enter the system through the interface. A static class diagram expresses dependencies between classes, not the temporal ordering of every event.

The Data Access Object pattern places persistence operations behind an interface such as `CustomerDAO`, `CartDAO`, `ItemDAO`, or `OrderDAO`. Repository classes implement these contracts. This arrangement makes the controller's required data operations explicit and permits a persistence mechanism to change without making the controller dependent on a specific storage API. The DAO abstraction concerns access to domain objects; it does not replace domain behavior in `Cart`, `Order`, or `Item`.

Table 2.1: Responsibilities represented by the design layers.

Layer or role	Principal responsibility	Representative classes
Interface (view)	Display data and expose user-facing actions.	CustomerView, CartView, ItemView, OrderView
Controller	Coordinate requests and use data-access contracts.	CustomerController, CartController, ItemController, OrderController
DAO contract	Specify retrieval and persistence operations.	CustomerDAO, CartDAO, ItemDAO, OrderDAO
Repository implementation	Realize a DAO contract using a persistence component.	CustomerDAOImpl, CartDAOImpl, ItemDAOImpl, OrderDAOImpl
Domain model	Represent customer, catalog, cart, order, payment, and shipment state.	Customer, Item, Cart, Order, Payment, Shipment

Chapter 3

E-Commerce Class Design

3.1 Overview of the final diagram

The design is arranged in three horizontal tiers. The upper package contains four views, each corresponding to a major user-facing area. The middle package contains four controller columns. Within each column, the controller, DAO interface, and repository implementation are vertically aligned so that their responsibilities can be compared directly. The lower model package contains the customer, order, and item subpackages. This layout preserves the project's original classes and relationships while making the principal layer boundaries easier to inspect.



Figure 3.1: Final MVC and DAO class design exported from the edited Visual Paradigm project. The complete image can be enlarged in the PDF viewer to inspect class members and dependencies.

3.2 Domain organization

The customer subpackage contains `Customer`, `FullName`, `Address`, and `Account`. It also includes `CustomerVIP` and `CustomerNew` as specializations of customer behavior or state. This arrangement keeps identity and personal information with the customer domain while allowing a specialized customer type to extend the base class.

The order subpackage contains `Cart` and `CartItem`, `Order` and `OrderItem`, and the associated `Payment` and `Shipment` classes. A cart is mutable preparation for a purchase; an order records the placed purchase. Keeping their item classes distinct allows an order line to retain order-time quantity and price rather than depending only on the current state of a cart or catalog item. The handout explicitly places order, payment, and shipment concepts together for this design stage [1].

The item subpackage contains the base `Item` and several catalog specializations, including `Book`, `Electronics`, `Clothes`, `Shoes`, `Laptop`, and `Mobile`. These specializations communicate variations in item attributes. A production design would still need to decide whether inheritance is preferable to category data for each item type; the diagram records the current educational design choice.

3.3 Representative interactions

1. **Customer lookup.** A customer-oriented action is coordinated by `CustomerController`. The controller depends on `CustomerDAO` for retrieval and updates. The DAO implementation uses a persistence component and returns a `Customer` for presentation by `CustomerView`.
2. **Cart modification.** `CartController` coordinates cart operations. It uses `CartDAO` and, where item data is required, `ItemDAO`. The domain model represents the cart and its `CartItem` entries, while the view presents the updated state.
3. **Catalog browsing.** `ItemController` uses `ItemDAO` to obtain items for `ItemView`. Item-specific attributes remain in the model instead of being embedded in the view.
4. **Checkout.** `OrderController` coordinates the transition from cart data to an `Order` and uses the relevant DAO contracts. `Payment` and `Shipment` are associated domain concepts. The diagram defines their class responsibilities but does not itself specify an atomic checkout transaction, payment authorization sequence, or shipping provider protocol.

These paths describe intended responsibilities. They are consistent with the dependency and realization structure visible in Figure 3.1, but a class diagram alone cannot prove the order of runtime messages. A sequence diagram and executable tests would be needed for that purpose.

Chapter 4

Evaluation and Limitations

4.1 Structural verification

The edited project retained all five class diagrams and the existing model elements. The final MVC diagram contains 35 class shapes. A bounds check found no pair of class rectangles overlapping within the same package, and the diagram’s connector endpoints refer to existing diagram elements. The SQLite container passed its integrity check, and Visual Paradigm exported Figure 3.1 from the edited project. These are reproducible checks of file structure, layout, and diagram export.

Table 4.1: Verification of the delivered design artifact.

Check	Observed result
Project database integrity	SQLite integrity check returned ok.
Diagram preservation	Five diagrams remained in the project.
Final-diagram class layout	35 class shapes; zero overlaps within the same package.
Connector references	No broken diagram-element endpoint references detected.
Visual Paradigm export	PNG generated from the edited <code>eComDesign.vpp</code> .

4.2 Interpretive limits and further work

The diagram supports a reasoned architectural review; it is not a running system. In particular, interface signatures do not establish input validation, transaction boundaries, error handling, concurrent inventory reservation, payment security, or persistence mapping. Further work would define those contracts precisely, map classes to a Spring implementation, and test representative workflows. A sequence diagram for checkout would be especially useful because it could specify the temporal relationship among cart validation, order creation, payment, and shipment scheduling.

Chapter 5

Conclusion

The final artifact expresses a clear progression from domain analysis to software design. Views, controllers, DAO contracts, repository implementations, and domain classes have distinct roles, while the customer, cart, order, and item areas remain traceable to the original e-commerce concepts. The revised layout makes the three-tier architecture and four functional controller columns easier to compare. Structural checks and a successful Visual Paradigm export support the integrity of the delivered UML artifact; implementation and runtime validation remain separate follow-on activities.

Bibliography

- [1] Course handout, *From Analysis to Design (Từ Phân Tích Đến Thiết Kế)*, four-page assignment PDF supplied for Information Systems Analysis and Design.
- [2] V. D. Hieu, *eComDesign.vpp*, Visual Paradigm class-diagram project and exported MVC design image, September 2026.